FX_GNN — Design Notes

Internal research log. Tracks architectural decisions, rejected approaches, and open questions. Not intended as polished documentation.

Node & Edge Encoding Rationale

Why dollar-centric?

Currencies operate in pairs, which creates a challenge: pair-wise feature engineering for country-specific factors (rates, sentiment) would be excessively complex and hurt interpretability. The chosen solution anchors all nodes to USD:

1. Pair-wise USD relationships exist for all major G10+ currencies — resolves data availability constraints.
2. The US is the central hub of global finance — there is a clear economic justification for understanding all currencies relative to the dollar.
3. Subsequent cross-currency relationships can be derived by cancelling the shared USD term. This can be isolated as a standalone algorithm later.
4. Non-currency nodes (rates, sentiment, equities, commodities) encode how American macro factors affect other currencies through their edges.

Design History

Rejected: Hierarchical Ensemble GNN

Original intention: An ensemble of inductive GNNs, each representing a distinct FX regime state. Contrastive loss would classify new observations by similarity to each regime-specific graph.

Why abandoned:

1. Sidelines the relational advantage of GNNs. Performance would depend more on how the `edge_matrix` was initialized than on what the transformer actually learned.
2. High overfitting risk — the model would likely learn to associate specific currencies with specific historical events rather than generalizing regime structure. Regularization, dropout, and augmentation would only partially mitigate this while adding computational cost.

Replaced with: A Graph Transformer architecture where edge computation is learned through unsupervised methods informed by FX domain priors. Backpropagation updates edges, not nodes. Then a temporal GRU is applied to capture relational evolutions across time.

Ideas Under Consideration

The following architectures have been noted as potentially relevant. None have been formally evaluated yet.

- **Line Graph NN → GNN stack** — convert edges to nodes for a two-stage relational model
- **EdgeConv** — dynamic edge convolution; suited for learning local geometric structure
- **EGNN** — equivariant GNN; preserves geometric constraints
- **R-GCN (Relational GCN)** — multiple relation types with type-specific weight matrices; aligns well with the multi-class edge design
- **TGN (Temporal Graph Network)** — event-driven temporal modeling; may suit tick-level or event-driven data
- **MTGNN (Multivariate Temporal GNN)** — designed for multivariate time series forecasting

MTGNN Notes

MTGNN is natively geared toward forecasting via entropy-based loss. Potential adaptations for regime classification:

- Swap the loss function for contrastive loss relative to prior states or aggregated regime representations
- Retain the forecasting objective and use auxiliary diagnostic metrics to assess regime transitions separately (without modifying the loss)

Primary limitation: The prediction-based loss function does not naturally express relational structure, making it a weaker fit for regime classification without modification.

Open Questions

- Formal definition of the n+1 edge relationship classes — which FX-specific frameworks to encode as priors (e.g., covered interest parity, purchasing power parity, carry relationships, risk-off/risk-on flows)?
- Handling weekend gaps for crypto nodes — simple forward-fill vs. masking vs. exclusion?
- TGT vs. Graph Transformer + GRU — run comparative experiment to assess interpretability tradeoff
- Curvature analysis — follow up with Michael Womack's masters thesis work on graph curvature as a potential input or diagnostic signal

---

Train/Validation/Test

- **2006-2020:** training set
- **2020-2022 (COVID):** testing set 1: evaluates transition detection — the sharpest regime transition in the dataset, never seen during training or validation
- **2023-2025:** testing set 2: evaluates out-of-sample generalization in a post-crisis normalization regime

The choice of 2 testing sets is intentional. Otherwise, conflating the two objectives into a single test set would obscure whether underperformance stems from poor transition detection or poor regime classification in unseen conditions.

The choice to omit a validation set is deliberate:

1. the limitation of the data set and importance of testing sets restricts the value of a validation set by nature of data use tradeoffs. To keep the two testing sets, we need to take from the training set which has around 3528 days which is already on the lighter end. If we only take a short interval for the validation set, that risks overfitting and cannot be trusted.
2. By nature of a graph transformer, the most important "hyperparameter" of how each feature relates to another is encoded by the transformer and engineered classification equations specified below. Thus, the primary hyperparameters of concern are the hyperparameters of the GRU. To resolve this concern, I will implement a distinct GRU on the 23 available features to transfer the dropout rate, learning rate, sequence length, hidden dimension, and count of layers. For hidden dimension and count of layers which aren't necessarily transferable, I will default to academia standards in typical implementations of GRUs on financial time-series data.

Alternative Future Direction

The Core Idea

A dual-headed architecture where the graph transformer produces two distinct levels of representations simultaneously: one node-level, one graph-level. Each fed into a different head with a different objective. The novel contribution is not the dual-head itself but the **coherence constraint** between the two heads that quantifies uncertainty in model forecasts while achieving prediction alongside transition probabilities.

Why Two Heads

The graph transformer output is split into two representations:

Node-level embeddings: pair-wise asset dynamics, fed into the GRU for next step prediction. Captures how individual relationships are evolving at the node level. The computations summarize into a pair-wise huber_loss metric, but it's not a loss function so it doesn't interfere with back prop, but the values update according to the global loss function which still ensures the data compounds.

Graph-level embedding: the entire graph collapsed into a single vector via pooling. Captures the holistic relational structure of the full graph at this timestep. Same set of logic here but instead of huber we'd use contrastive learning to quantify the level of similarity/differences present.

Thesis: the purpose of using these huber_loss and contrastive loss as metrics is to compute a global loss function that forces these two to agree with a set of hyperparameter or learn the weight of how to allocate these two. This property means the algorithm can make classificaitions and understand transitions. More importantly, the direction and degree of similarity and classification changes should agree as an internal metric to quantify the uncertainty present within this model. This is the loss function that exists throughout the graph architecture. Then each layer of the graphs serve as a snapshot of a sequence entered into the GRU.
So, the GRU will be able to pick up on the way the agreements between dissimilarity and classification to unify them in making predictions given the level of agreement between these two values.

References

- FX Graph Learning for Statistical Arbitrage — ACM 2024
- Michael Womack — curvature analysis (masters thesis, citation TBD)

Future Architecture Customizations

Loss Function

- Probabilistic output head — edge_mean and edge_log_var per edge
- β-NLL (beta=0.5) replacing HuberLoss on edge predictions
- Fisher-z transformation on edge targets before loss computation
- Attention entropy as auxiliary term with gradient-balanced lambda
  loss = beta_nll + lambda * (-attention_entropy.mean())
- Curriculum training — prediction only first, entropy term in Phase 2

Node Features

- Beta and residual features — USD sensitivity decomposition
  input_dim becomes 23 × 4 = 92
- GARCH conditional volatility per node
- Robust IQR scaling replacing z-score normalization

Edge Design

- Edge type specific normalization — within type not universal
- Separate encoder per node type before shared attention
- Hurst exponent per pair — mean reversion vs momentum prior
  determines Layer 2 prior type per edge

Inference Diagnostics

- GDN median/IQR robust deviation scoring per edge
- Triangulation source attribution across G10 neighbors
- Duration estimation via Ornstein-Uhlenbeck half-life per edge

Data

- Options and swaps — deferred pending edge prior engineering

---

Engineering Log

4-28 - Architecture

Change to Structural Design

- With an abundance of nodes across different categories, it's best to optimize edges rather than nodes. The nodes are standardized and interconnected via the US dollar. This way, it connects the multimodal nodes and helps derive higher-dimensional understandings of relationships beyond just strictly FX, but also relationships to sentiment, vol, macroecon data, etc.

05-01 - Data Source

FRED Data Correction

- 'SPX': the ID for S&P on Fred is weekly. The algo is US-centric, and missing S&P data at high frequencies is missing a core proponent of the signal.
  Consider - Stooq or yf with delayed queries
- HY/IG spreads: limited data availability on Fred, insufficient to cover the training set.
  Consider - direct computation from Fred data.
- GOLD/RUT/Copper: no longer available on Fred, move to yf query with rate limits.

05-02 - Data Source

G10 Data Selection

- Will omit NZD and NOK data due to inconsistency and limited availability during targetted training set duration from 2000-2022.

05-05-2026 - Architecture

Loss Function

- Huber_loss: it optimizes the balance between small and larger errors in a way that's superior to MAE or MSE individually. It's the loss_function of choice to optimize the GRU.
- GRU hyperparameter tunning: since the algorithm will not have a validation set, it's important to scout the hyperparameter sizes of the GRU with alternative methods. I plan to implement a distinct GRU on the 23 features to tune optimal and generalizable hyperparameters.
- Ground-truth engineering: I realized there's no set system that quantifies a historical set of "regimes" from known financial metrics. This alone could be a direction to explore and publish data sets for available use.

05-06-2026 - Objective/Architecture

Node/Graph Level

- After further literature review, FX regimes (defined as global relationships i.e. macro/holistic shifts) occur rarely and are frequently event driven. To add to the value of this model, the focus will now shift towards forecasting node-wise relational convergences and divergences.
- Additionally, traders should take an inference time technique to threshold changes between different nodes. Then use additional currency nodes in G10 as points of reference to deduct the root cause i.e. which country is likely the driver of a relational shift.
  EX: EUR/USD vs JPY/USD relationship shifted, we can check the degree of relational shifts between EUR and JPY with CAD or GBP: the model would detect these anomlies and pair-wise shifts in addition to EUR and JPY, but it helps with interpretability and decision making regarding:

1. whether the shift is localized
2. the country of origin driving the shift

- Triangulation, validating the scope of relational breaks in n pairs with m-n pairs, is an interpretability tool only. The model already detects both localized and systemic deviations from its output matrix directly.

Data

- Options: we will add derivatives for the 8 currencies along with crypto-currencies in the form of swaps and options.

Use Case

- With an increased emphasis on pair-wise relational forecasts at the node-level rather than the graph level, it fulfill regression functions towards statistical arbitrage. As the graph transformer outlines the direction of convergences and divergences at present, each snapshot of the graph inputted into the GRU will give a relational forecast on the likelihood of convergence, divergence, or stability.
- This methodology is sufficient to fulfill the objective of statistical arbitrage were deviations and convergences serves as signals for entry/exit. Meanwhile, forecasts of different time intervals is representative of larger macro shift.
- TODO: after completion of the core architectures, it's likely that I will add a graph level evaluation of how much macro regime shifts has occured and how "attributable" relational breaks is caused by macro signals.

Architecture

- The loss function of choice is still huber_loss. The objective of the algorithm requires classification of particular classes to determine whether a pair-wise relationship might diverge or converge. But I will engineer a similarity score and an uncertainty score in addition to this directional classification output.

1. similarity score: a function akin to contrastive learning without back prop.
   Consider: cos_sim = F.cosine_similarity(h_t, h_t_rolling_mean, dim=-1)
   dissimilarity = 1 - cos_sim
2. uncertainty score: a function that computes the "agreement" between the shifts in different edges. This score will likely be a product of engineered class specific relationships as part of "layer 2" of edge relational architectures.

05-07-2026 Architecture

Loss Function

- We have ~3400 time-steps of data for each edge that exists across 23 nodes. We care about node-level pair-wise relationships so it's condensed at 3400. That's insufficient to train 4 distinct loss_terms with weights for discrete lambda 1-4. Alternatively, preserve these loss functions weighted: W1(loss_pred) + W2 (loss_discrepency).

1. loss_pred is a classification term
2. loss_discrepency is a term to determine similarity.

Feature Engineering

- I can enrich features by adding beta/residual composition i.e. CAD/USD uses USD as denomination. We also have DXY (flat US currency), if we take the same time interval between CAD/USD and DXY, we can eliminate the denomination of CAD. What's left is a "second order" sensitivity to shifts in US currency which we can derive averages of or build analysis from.

05-08-2026 Data

training/testing set

- After initial examination, the baseline_GRU performed significantly below standards to be usable. It reveals how unique COVID and recovery is as a regime. Thus, the GRU's prediction error reaching as high as 1.4 stds implies this baseline has failed as an industry standard baseline. Similarly, the target architecture would likely similarly fail to generalize onto COVID.
- Thus, the design choice is to include 2020-2023 as part of the training set and retain only 2023-2025 as the testing set.

Addressing Limited Out-of-Sample Data

- 2023-2025 (~750 trading days) is insufficient for robust generalization claims from a single split. Three techniques address
  this.

- CPCV: divide 2006-2025 into 8 chronological groups, test every combination of 2 groups as holdout — 28 distinct OOS paths, a
  distribution of metrics rather than a single number. Purging at graph snapshot level prevents temporal and relational leakage.Embargo = seq_len + forecast_horizon after each fold.

- Regime-Conditional Evaluation: 3-state HMM fit on 2006-2022 (VIX + broad FX index) labels 2023-2025 by regime. All metrics
  reported per regime, not averaged. Qualitative anchors: SVB (March 2023), JPY carry unwind (July 2024), Fed pivot (Sept 2024).

- Vector Block Bootstrap: Politis-Romano stationary bootstrap across all 23 nodes simultaneously, preserving cross-asset
  correlation. 1000 replicates produce confidence intervals on all metrics. Block length via Politis-White on PC1 of return matrix.
  Limited to stable-period metrics only.

  NOTE: these techniques make the evaluation defensible, not perfect. Generalization claims remain bounded by 1-2 crisis events per
  regime type in 19 years of daily data.

5-10-2026 Architecture

loss function

- the loss function decision is ultimately to use huber loss as it takes an optimal middle ground between MSE and MAE
- the 3 layered architecture:

1. unsupervised transformer mechanism, computes inherent weights
2. a set of engineered equations that capture different cross-node relationships
3. a loss function that computes composite alignment between layers 1 and 2 while being able to represent the initial relationship at hand.

The challenge is with step 3. The engineered equations might have a different "measure" as its outputs in contrast to the unsupervised transformer's outputs. While an additional attention mechanism that gets concat later is a sufficient relational and certainty representation of these edges, it makes interpretability nearly impossible and computations potentially inaccurate as the measures are distinct.

Temporary solution: construct layer 1 exclusively as unsupervised. It's the only layer receiving back prop update. Layer 2 is fixed and computed at every turn with data at that particular edge. It can get concat into the weight features, but it won't receive back prop updates. It contributes to prediction accuracy and can be used with attention mechanism (after standardization with linear projection), for inference time validation of predictive agreement.

5-11-2026 Architecture

Overfitting Regulation

- Graph Transformer: a snapshot a day: theoretically 253 edges available which overwhelms daily data input

1. Transformer details: it relates 2 edges, 2 layers upon forward pass. In total, it relates to 4 edges total, but the last 2 edge relations occured after the first layer, so those two edges are similarly edge_prime such that they've received enrichment from their proximate neighbors
2. Masking: we mask all features below the upper-quartile.
3. Layer 2 and 3: we encode equations to represent economic relationships which are fixed. Layer 3 then relates layers 1 and 2 into a composite function. Layer 2 achieves a regularizing function while encoding additional frameworks that may be difficult for fully supervised algorithms to discover.

5-13-2026 Architecture

Overfitting Regulation - Graph

- I will have 4 attention nodes, 2 layers of full Graph Transformer communication, and preserve most significant 25% of edges built.
- It limits oversmoothing as each edge would only access inputs from 5-6 others such that vectors wouldn't lose their "original identities." I would've liked to preserve less, but the currencies are US-centric i.e. currency pairs denominated by USD. That means the US specific features would likely dominate, thus we need 5-6 to ensure each edge, at least, encodes some pair-wise currency relation.
- It avoids overfitting due to limited degrees of freedom from the constraints edges available.
- Winsorization applied to the distortion of QKV values

Overfitting Regulation - GRU

- Graph vector bootstrapping as per the Politis Romano method will be implemented to enrich the intervals of regimes available to sample from. Note: it assumes the graph transformer has extracted

genuine signals and denoised the dataset to sufficient degree to be optimal.
- I will compute a GRU per particular edge as to avoid overfitting general graphical functions.
- heavy conventional regularization metrics: aggressive dropout, weight-decay ADAM, early stopping, etc.
- Multi-horizon forecasts will be made such that we encompass predictions on (1, 5, 10) day intervals.

Data

- Use case specification of this algorithm for arbitrage requires an understanding that CAD/JPY =/= weighted version of (JPY/USD, CAD/USD). The latter involves an implicit level fo sensitivity exposure to USD that's not present within the intrinsic weights of CAD/JPY.

5-14-2026 Fundamental Reorganization

Pair-wise Reorganization

- I realized if the purpose of the algorithm was to fulfill node-level predictions on shifts and divergences in currency relationships for arbitrage, there is no reason to include all G10 currencies at all and build a composite graph representative of all currencies. Rather, it's likely best to build a condensed version that only includes pairs we care about. It limits the objective which can likely improve interpretability and constrain overfitting. Most importantly, it avoids needing to have trained on a large sample only to have tossed out majority of the edges.